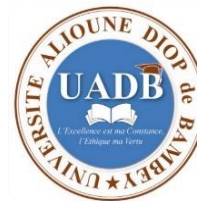




REPUBLIQUE DU SENEGAL



UNIVERSITE ALIOUNE DIOP BAMBEY

UFR : SATIC

Master 1 Système d'Information

PROJET DE FIN MODULE

VIRTUALISATION ET CLOUD COMPUTING

Déploiement d'une architecture réseau 3-tiers
Virtualisé sur OpenShift



Présenté par :

Gana Faye

Professeur :

Dr. Babou

Année académique : 2025-2026

Table des matières

Liste des figures :	3
Listes des tableaux :	4
1. INTRODUCTION.....	5
2. ARCHITECTURE DE LA SOLUTION	7
2.4. Règles de sécurité (Firewall).....	9
3. MISE EN ŒUVRE TECHNIQUE.....	12
3.3. Déploiement des machines virtuelles	19
3.4. Déploiement de la base de données MySQL	22
3.5. Déploiement de l'application Node.js sur le vm-web.....	24
4. AUTOMATISATION GITOPS	26
4.2. Intégration continue / Déploiement continu	28
5. TESTS ET VALIDATION.....	30
6. RÉSULTATS OBTENUS.....	32
7. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS	34
8. CONCLUSION ET PERSPECTIVES	35
8.1. Bilan du projet.....	35

Liste des figures :

Figure 1: Architecture du réseau sur le cluster open shift	7
Figure 2 : Configuration du réseau avec iptables	8
Figure 3: Règles iptables sur VM-WEB	10
Figure 4 : Structure du dossier du projet	12
Figure 5 : Workflow GitHub Actions (deploy.yaml)	14
Figure 6: Configuration des secrets variables.....	16
Figure 7: configuration des reseaux lan et du resaux dmz	18
Figure 8: Templates YAML du VM-GW.....	19
Figure 9: Fichier déploiement Yaml VM-WEB	20
Figure 10: Fichier déploiement du pod DB mysql	22
Figure 11: Initiation et configuration du Db MySQL	23
Figure 12 : Fichier config du déploiement de l'application sur le VM-Web	24
Figure 13 : Code du page d'accueil de l'application web encapsulé sur le serveur	25
Figure 14 : AUTOMATISATION GITOPS.....	27
Figure 15 : Validation du CI/CD Pipeline.....	28
Figure 16: Tests de validation du pipeline	29
Figure 17 : image de l'application déployer	30
Figure 18 : Génération de la route externe pour l'application soit accessible aux users.....	30
Figure 19 : Vérification des ressources sur le cluster open shift	32

Listes des tableaux :

Tableau 1: architectures 3-Tiers.....	5
Tableau 2 : Les objectifs.....	6
Tableau 3 : Approche GitOps.....	6
Tableau 4 : Composants et roles	7
Tableau 5 : Composants Techniques	8
Tableau 6 : Adresses IP utilisées	8
Tableau 7 : Détails des adresses IP	8
Tableau 8 : Etapes de déploiement	15
Tableau 9 : Les variables secrets.....	16
Tableau 10 : Caractéristiques principales	19
Tableau 11 Caractéristiques Techniques du VM-WEB.....	21
Tableau 12 : Caractéristiques du Pod DB mysql	22
Tableau 13 : Status des vm et pod.....	29
Tableau 14 : Synthèse du pipeline.....	31
Tableau 15 : Infrastructure déployée	32
Tableau 16 : Les difficultés rencontrés et solutions	34
Tableau 17 : Compétences acquises	35

1. INTRODUCTION

1.1. Contexte du projet

- Objectifs du module Cloud Computing et Virtualisation

Le module **Cloud Computing et Virtualisation** s'inscrit dans une formation axée sur les technologies modernes d'infrastructure. Il vise à nous former aux concepts fondamentaux du cloud, à la virtualisation des ressources, ainsi qu'à l'automatisation des déploiements.

L'un des enjeux majeurs du cloud est la capacité à déployer des applications robustes, sécurisées et évolutives. L'architecture **3-Tiers** (trois niveaux) est un modèle de référence dans ce domaine. Elle organise l'infrastructure en trois couches distinctes :

- Importance des architectures 3-Tiers dans les environnements d'entreprise

Niveau	Rôle	Exemple dans notre projet
Présentation (Web)	Interface utilisateur	VM-WEB avec application Node.js
Application (Métier)	Logique métier	Application Node.js
Données (Base)	Stockage et gestion des données	MySQL

Tableau 1: architectures 3-Tiers

Cette séparation offre de nombreux avantages :

- **Sécurité** : isolation des couches via une segmentation réseau (DMZ/LAN)
- **Maintenabilité** : chaque couche peut être modifiée indépendamment
- **Scalabilité** : possibilité de dimensionner chaque niveau séparément

1.2. Objectifs du projet

Ce projet a pour objectif de déployer une **infrastructure 3-Tiers complète** sur la plateforme **OpenShift Virtualization** et j'ai adopté l'approche **GitOps**.

Les objectifs spécifiques sont :

Objectif	Description
Déployer une architecture 3-Tiers	Mettre en place trois composants : passerelle/firewall, serveur web et base de données
Segmenter le réseau	Créer des zones DMZ (192.168.100.0/24) et LAN (192.168.10.0/24)
Configurer un firewall	Utiliser iptables sur la passerelle pour filtrer les flux entre les zones
Déployer une application	Créer une application Node.js connectée à une base de données MySQL

Tableau 2 : Les objectifs

➤ Mon approche : GitOps

Synchronisation du déploiement avec le dépôt distant.

Objectif	Description
Infrastructure as Code (IaC)	Versionner tous les fichiers de configuration dans GitHub
Automatisation	Utiliser GitHub Actions pour déployer automatiquement l'infrastructure
Reproductibilité	Permettre de recréer l'infrastructure à l'identique en un simple git push

Tableau 3 : Approche GitOps

2. ARCHITECTURE DE LA SOLUTION

L'architecture déployée suit le modèle **3-Tiers** (trois niveaux) avec une segmentation réseau stricte entre les zones **DMZ** (Demilitarized Zone) et **LAN** (Local Area Network).

2.1. Vue d'ensemble de l'architecture



Figure 1: Architecture du réseau sur le cluster open shift

■ Composants et leurs rôles

Composant	Type	Rôle principal
VM-GW	Machine virtuelle	Passerelle réseau et firewall. Assure le routage entre les zones DMZ et LAN. Filtre le trafic selon des règles iptables.
VM-WEB	Machine virtuelle	Héberge l'application Node.js. Située dans la zone DMZ, elle est accessible depuis Internet via la passerelle.
MySQL	Pod Kubernetes	Base de données relationnelle. Située dans la zone LAN, elle n'est accessible que par l'application (VM-WEB et Node.js).
Node.js App	Encapsulé sur le VM-Web	Application web développée avec Express.js. Se connecte à MySQL pour lire et écrire des messages. Exposée publiquement via une route HTTPS.

Tableau 4 : Composants et rôles

2.2. Composants techniques

Composant	Rôle	Technologie	Version
VM-GW	Passerelle / Firewall	Ubuntu 22.04 LTS + iptables	1.8.7
VM-WEB	Serveur Web	Ubuntu 22.04 LTS + Nginx + Node.js	Node.js 18
MySQL	Base de données	MySQL Community Server	8.0
Node.js App	Application web	Node.js + Express.js	Node.js 18
Orchestration	Plateforme de conteneurs	OpenShift	4.12+
Virtualisation	Gestion des VMs	KubeVirt	1.7.1

Tableau 5 : Composants Techniques

2.3. Adresses IP utilisées

Conformément au cahier des charges, les plages d'adresses suivantes ont été utilisées pour la segmentation réseau :

Zone	Réseau	Passerelle	Hôtes	Description
DMZ	192.168.100.0	192.168.100.1	192.168.100.10	Zone démilitarisée accueillant le serveur web
LAN	192.168.10.0	192.168.10.1	-	Zone interne accueillant la base de données

Tableau 6 : Adresses IP utilisées

Détail des adresses affectées

Machine	Interface	Adresse IP	Zone
VM-GW	lo:1	192.168.100.1	DMZ (passerelle)
VM-GW	lo:2	192.168.10.1	LAN (passerelle)
VM-WEB	lo:1	192.168.100.10	DMZ

Tableau 7 : Détails des adresses IP

Note technique : En raison des restrictions du Dev Sandbox (Multus non disponible), ces adresses ont été configurées sur des interfaces virtuelles (lo:1, lo:2) dans les VMs. La segmentation logique est maintenue via les règles iptables.

```
(base) gana-faye@gana-faye-ThinkPad-T14s-Gen-1:~/Bureau/CoudComp_Virtualisation/projet
-coud-Computing-Virtualis virtctl console vm-gw -n ganafaye-dev
# Login: ubuntu / Password: ubuntu

# Vérifier les IPs _
target    prot opt source                destination            tcp dpt:3306
ACCEPT    tcp  --  192.168.100.0/24        192.168.10.0/24        tcp dpt:80
ACCEPT    tcp  --  0.0.0.0/0              192.168.100.0/24        tcp dpt:443
Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination

(base) gana-faye@gana-faye-ThinkPad-T14s-Gen-1:~/Bureau/CoudComp_Virtualisation/projet
-coud-Computing-Virtualis virtctl console vm-web -n ganafaye-dev
# Login: ubuntu / Password: ubuntu

# Vérifier l'IP
ip addr show | grep 192.168
    inet 192.168.100.10/24 scope global lo
    ubuntu@vm-w# Tester MySQL MySQL
mysql -h mysql-service.ganafaye-dev.svc.cluster.local -u webapp -ppassword -e "SHOW DA
DATABASES;"
Command 'mysql' not found, but can be installed with:
sudo apt install mysql-client-core-8.0 # version 8.0.45-0ubuntu0.22.04.1, or
sudo apt install mariadb-client-core-10.6 # version 1:10.6.23-0ubuntu0.22.04.1
ubuntu@vm-web:~$
```

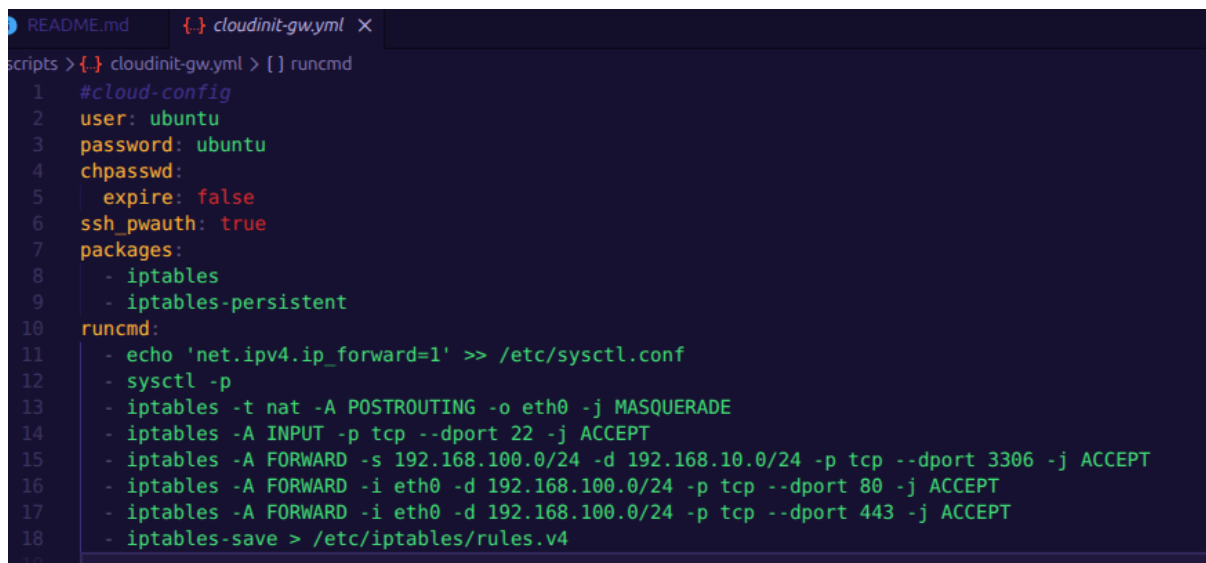
Figure 2 : Configuration du réseau avec iptables

2.4. Règles de sécurité (Firewall)

- Règles iptables sur VM-GW

Analyse des commandes :

- ✓ **Activation du Routage (IP Forwarding)** : Les lignes 11 et 12 (`net.ipv4.ip_forward=1`) transforment la VM en un véritable routeur capable de faire passer le trafic entre les différents sous-réseaux.
- ✓ **NAT / Masquerading** : La ligne 13 permet aux machines du LAN (192.168.100.0/24) de sortir sur Internet en utilisant l'adresse IP publique de la VM Passerelle.
- ✓ **Sécurisation (Filtrage) : Ligne 15** : Autorise spécifiquement le trafic SQL (port 3306) entre la DMZ et le LAN.
- ✓ **Lignes 16-17** : Autorise le trafic Web (80/443) entrant vers la DMZ (192.168.100.0/24).
- ✓ **Persistance** : La ligne 18 assure que ces règles survivront à un redémarrage de la machine.



```
scripts > cloudinit-gw.yml > [ ] runcmd
1 #cloud-config
2 user: ubuntu
3 password: ubuntu
4 chpasswd:
5   expire: false
6 ssh_pwauth: true
7 packages:
8   - iptables
9   - iptables-persistent
10 runcmd:
11   - echo 'net.ipv4.ip_forward=1' >> /etc/sysctl.conf
12   - sysctl -p
13   - iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
14   - iptables -A INPUT -p tcp --dport 22 -j ACCEPT
15   - iptables -A FORWARD -s 192.168.100.0/24 -d 192.168.10.0/24 -p tcp --dport 3306 -j ACCEPT
16   - iptables -A FORWARD -i eth0 -d 192.168.100.0/24 -p tcp --dport 80 -j ACCEPT
17   - iptables -A FORWARD -i eth0 -d 192.168.100.0/24 -p tcp --dport 443 -j ACCEPT
18   - iptables-save > /etc/iptables/rules.v4
```

Figure 3 : Fichier de configuration cloudinit-gw.yml pour l'automatisation du routage et des règles de pare-feu (IPtables)

Descriptions de la configuration :

Cette capture d'écran présente le fichier de configuration **Cloud-Init** utilisé pour le déploiement automatisé de la **VM Passerelle** (Gateway).

Ce script d'automatisation (IaC - Infrastructure as Code) remplit trois fonctions critiques pour la sécurité du projet :

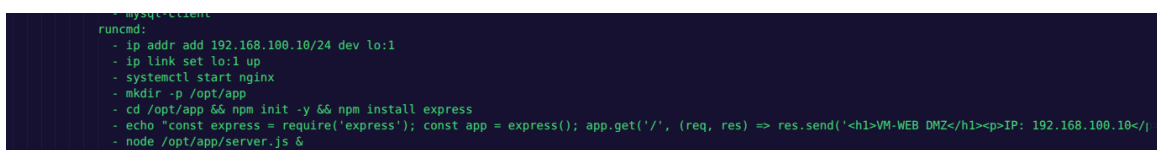
1. **Provisionnement des utilisateurs** : Il configure l'accès SSH et définit les identifiants par défaut (ubuntu/ubuntu) pour l'administration de la machine.
2. **Configuration Réseau (Routage)** : Il active l'**IP Forwarding** au niveau du noyau Linux, transformant la machine virtuelle en un routeur capable de diriger le trafic entre le réseau externe (Internet) et les réseaux internes (DMZ et LAN).
3. **Pare-feu et NAT (IPtables)** : * Il met en place le **Masquerading** (NAT) pour permettre aux machines internes de communiquer vers l'extérieur.
 - Il définit des règles de filtrage strictes : autorisation du flux **SQL (port 3306)** entre les segments et ouverture sélective des ports **HTTP/HTTPS (80/443)** pour l'accès aux services web.

L'utilisation de iptables-persistent garantit que l'ensemble de cette politique de sécurité est automatiquement rechargé à chaque démarrage de la VM

Avantage :

- ✓ Au lieu de configurer le réseau manuellement, j'ai utilisé un script reproductible. Ce qui permet d'automatiser la configuration lors du push vers mon GitHub et Git Actions vas directement configurer comme suit sur open shift ce qui est un gain de temps.

- **Règles iptables sur VM-WEB**



```
mysql-client
runcmd:
- ip addr add 192.168.100.10/24 dev lo:1
- ip link set lo:1 up
- systemctl start nginx
- mkdir -p /opt/app
- cd /opt/app && npm init -y && npm install express
- echo "const express = require('express'); const app = express(); app.get('/', (req, res) => res.send('<h1>VM-WEB DMZ</h1><p>IP: 192.168.100.10</p>');"
- node /opt/app/server.js &
```

Figure 3: Règles iptables sur VM-WEB

Description du script cloud-init (Serveur Web)

Cette capture d'écran détaille la section `runcmd` du fichier de configuration Cloud-init pour le **Serveur Web**. Ce script automatise la mise en service de l'application dès le premier démarrage de la machine virtuelle.

Analyse des commandes :

1. Configuration Réseau Locale (`ip addr add`) :

- Le script assigne l'adresse IP 192.168.100.10/24 à une interface de loopback (lo:1). Cela permet de simuler ou de fixer l'identité réseau interne du serveur de manière persistante.

2. Gestion des Services (`systemctl start nginx`) :

- Le serveur **Nginx** est démarré pour servir de serveur proxy ou pour gérer les requêtes HTTP entrantes.

3. Déploiement de l'Application Node.js :

- **Préparation** : Création du répertoire `/opt/app` et initialisation d'un projet Node.js (`npm init`).
- **Installation** : Installation du framework **Express**, utilisé pour créer le serveur web minimaliste.
- **Génération du code** : Une commande `echo` crée dynamiquement le fichier `server.js`. Ce fichier contient une route unique renvoyant un titre HTML "VM-WEB DMZ" et l'adresse IP du serveur.

4. Exécution :

- L'application est lancée en arrière-plan avec `node /opt/app/server.js &`, rendant le service immédiatement disponible pour les tests de connectivité depuis la passerelle ou l'utilisateur final.

Rôle dans l'architecture :

Ce script prouve l'automatisation du déploiement de la couche applicative. Il garantit que chaque instance du serveur web est configurée de manière identique, sans intervention manuelle, respectant ainsi les principes du **Cloud Native** et du **DevOps**.

3. MISE EN ŒUVRE TECHNIQUE

3.1. Infrastructure as Code (GitHub Actions)

- Structure et Analyse du projet ou du dépôt GitHub

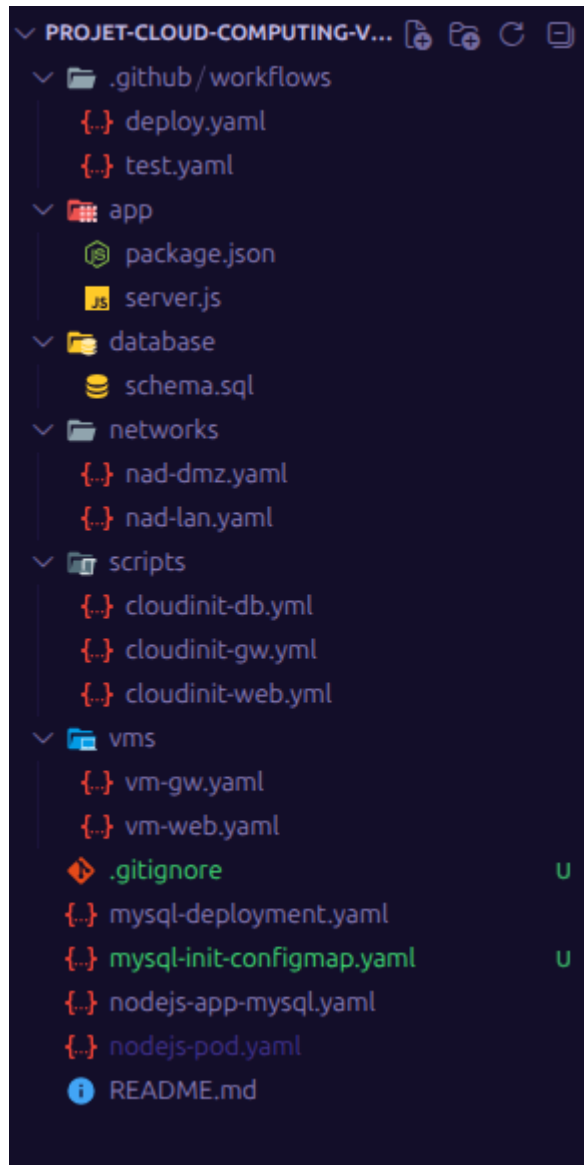


Figure 4 : Structure du dossier du projet

L'organisation des fichiers confirme une approche **Infrastructure as Code (IaC)**. Voici le détail des répertoires clés correspondant à votre diagramme :

1. .github/workflows (Le moteur CI/CD)

- `deploy.yaml` et `test.yaml` : Ces fichiers contiennent les instructions pour **Github Actions**. Ils automatisent les tests de l'application et son déploiement sur l'infrastructure (OpenShift/VMs) dès qu'un changement est détecté sur Github.

2. app/ (La couche Applicative)

- On y retrouve `server.js` et `package.json`. C'est le code de l'application Node.js qui sera exécuté sur le **Serveur Web** ou dans le **Pod Node.js** sur OpenShift.

3. database/ (La couche Persistance)

- `schema.sql` : Contient la structure des tables pour la base de données **MySQL**.

4. networks/ (La couche Réseau)

- `nad-dmz.yaml` et `nad-lan.yaml` : Ces fichiers définissent probablement les *Network Attachment Definitions*. Ils permettent de segmenter le réseau entre la **DMZ** (où se trouve le serveur web) et le **LAN** interne (où se trouve la BD).

5. scripts/ (L'automatisation du provisionnement)

Ce dossier contient les scripts de configuration automatique au démarrage (Cloud-init) :

- `cloudinit-gw.yml` : Configuration de la **Passerelle (VM1)**.
- `cloudinit-web.yml` : Configuration du **Serveur Web** (celui vu dans l'image précédente).
- `cloudinit-db.yml` : Configuration du **Serveur de Base de Données**.

6. vms/ (Définition des Machines Virtuelles)

- `vm-gw.yaml` et `vm-web.yaml` : Définitions YAML pour créer les VMs sur une plateforme comme KubeVirt ou OpenShift Virtualization.

7. Fichiers à la racine (Déploiement Kubernetes/OpenShift)

- `mysql-deployment.yaml` et `nodejs-app-mysql.yaml` : Fichiers de déploiement pour orchestrer les conteneurs et assurer la liaison entre l'application et la base de données.

Synthèse

J'ai choisi cette arborescence car elle est très propre. Elle sépare clairement :

1. **Le Code** (app/)
2. **L'Infrastructure** (vms/, networks/)
3. **La Configuration** (scripts/)
4. **L'Automatisation** (.github/workflows/)

- **Workflow de déploiement**

Analyse du Workflow GitHub Actions (deploy.yaml)

```
name: Deploy Infrastructure

on:
  push:
    branches: [ main ]
    paths:
      - 'vms/**'
      - 'mysql-deployment.yaml'

env:
  OPENSIFT_SERVER: ${ secrets.OPENSIFT_SERVER }
  OPENSIFT_TOKEN: ${ secrets.OPENSIFT_TOKEN }
  NAMESPACE: ganafaye-dev

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Install oc
        run: |
          curl -LO https://mirror.openshift.com/pub/openshift-v4/clients/ocp/latest/openshift-client-linux.tar.gz
          tar xzf openshift-client-linux.tar.gz
          sudo mv oc /usr/local/bin/

      - name: Install virtctl
        run: |
          curl -LO https://github.com/kubevirt/kubevirt/releases/download/v1.7.1/virtctl-v1.7.1-linux-amd64
          chmod +x virtctl-v1.7.1-linux-amd64
          sudo mv virtctl-v1.7.1-linux-amd64 /usr/local/bin/virtctl

      - name: Login
        run: |
          oc login --server=${ env.OPENSIFT_SERVER } --token=${ env.OPENSIFT_TOKEN } --insecure-skip-tls-verify=true
          oc project ${ env.NAMESPACE }

      - name: Deploy VMs
        run: |
          oc apply -f vms/vm-gw.yaml
          oc apply -f vms/vm-web.yaml

      - name: Deploy MySQL
        run: oc apply -f mysql-deployment.yaml

      - name: Start VMs
        run: |
          /usr/local/bin/virtctl start vm-gw -n ${ env.NAMESPACE } || true
```

Figure 5 : Workflow GitHub Actions (deploy.yaml)

Ce fichier définit l'automatisation du déploiement de mon infrastructure sur OpenShift. Il utilise une approche **GitOps** : toute modification du code déclenche une mise à jour de l'infrastructure.

1. Déclencheur (Trigger)

Le workflow se lance automatiquement lors d'un push sur la branche main, mais uniquement si des fichiers dans les répertoires suivants sont modifiés :

- Vms / (les définitions de vos machines virtuelles).
- mysql-deployment.yaml (la configuration de la base de données).

2. Environnement et Sécurité

Le script utilise des variables d'environnement et des **Secrets GitHub** pour se connecter de manière sécurisée à votre cluster :

- OPENSHIFT_SERVER : L'adresse de votre console OpenShift.
- OPENSHIFT_TOKEN : Le jeton d'authentification (pour éviter de mettre des mots de passe en clair).
- NAMESPACE : ganafaye-dev : Le projet spécifique sur OpenShift où les ressources seront créées.

4. Les Étapes du Job (deploy)

Étape	Action	Rôle dans le projet
Install oc	Télécharge et installe le client OpenShift (oc).	Permet d'envoyer des commandes au cluster.
Install virtctl	Installe l'outil de contrôle KubeVirt (virtctl).	Nécessaire pour gérer le cycle de vie des VMs (démarrage/arrêt).
Login	Connexion au cluster avec le token.	Établit la session sécurisée vers ganafaye-dev.
Deploy VMs	oc apply -f vms/vm-gw.yaml et vm-web.yaml.	Crée ou met à jour la Passerelle et le Serveur Web du diagramme.
Deploy MySQL	oc apply -f mysql-deployment.yaml.	Déploie l'instance de base de données MySQL .
Start VMs	virtctl start vm-gw et vm-web.	Force le démarrage des machines virtuelles après leur création.

Tableau 8 : Etapes de déploiement

Lien avec le Diagramme d'Architecture

- **La flèche "Github Actions" → "OpenShift"** : C'est exactement ce que fait ce fichier via la commande `oc login` et `oc apply`.
 - **La VM1 (Passerelle) et Serveur Web** : Ils sont instanciés par l'étape "Deploy VMs".
 - **Le Serveur BD / MySQL** : Il est instancié par l'étape "Deploy MySQL".
-
- **Secrets GitHub configurés**

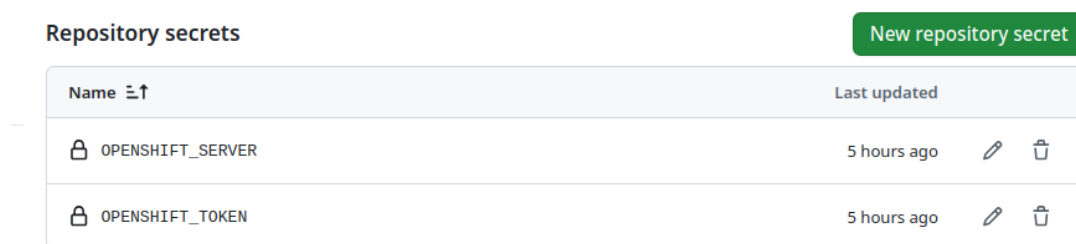


Figure 6: Configuration des secrets variables

Configuration de la Sécurité (GitHub Secrets)

Pour permettre à GitHub de communiquer avec OpenShift sans exposer de données sensibles, les secrets suivants ont été configurés dans le dépôt (comme vu dans la capture d'écran) :

Nom du Secret	Description	Utilisation dans le YAML
OPENSIFT_SERVER	L'URL de l'API du cluster OpenShift.	<code>{{ secrets.OPENSIFT_SERVER }}</code>
OPENSIFT_TOKEN	Le jeton d'accès (SA Token) avec les droits d'édition sur le namespace.	<code>{{ secrets.OPENSIFT_TOKEN }}</code>

Tableau 9 : Les variables secrets

5. Détails du Workflow de Déploiement

Le fichier `.github/workflows/deploy.yaml` orchestre les étapes suivantes :

A. Préparation de l'environnement de build

Le runner utilise `ubuntu-latest` et installe les outils nécessaires :

- **oc client** : Pour les commandes standards OpenShift (`oc apply`, `oc login`).
- **virtctl** : Extension spécifique pour gérer les machines virtuelles via KubeVirt.

B. Déploiement des Composants

Le script applique les fichiers de configuration Kubernetes/OpenShift :

- **VMs** : `vms/vm-gw.yaml` (Passerelle avec `pfSense/iptables`) et `vms/vm-web.yaml`.
- **Base de données** : `mysql-deployment.yaml`.

C. Gestion du Cycle de Vie

Une fois les définitions appliquées, le workflow force le démarrage des services :

```
virtctl start vm-gw -n ganafaye-dev || true
```

```
virtctl start vm-web -n ganafaye-dev || true
```

6. Correspondance avec l'Architecture Cible

- **Zone Publique** : La VM "Passerelle" est configurée pour gérer le trafic entrant via un pare-feu (`pfSense/iptables`).
- **Zone DMZ/Interne** : Le "Serveur Web" communique avec le "Serveur BD" (MySQL) via le réseau interne du cluster.
- **Automatisation** : GitHub Actions agit comme l'opérateur distant qui maintient l'état désiré de cette infrastructure.

***Inconvenant** : il faut assurer que le Token OpenShift n'expire pas, sinon le workflow échouera avec une erreur "Unauthorized".*

3.2. Déploiement des réseaux

L'architecture utilise deux interfaces réseaux distinctes pour une sécurité accrue :

Réseau DMZ (dmz-network)

- **Usage** : Exposition du serveur Web.
- **Bridge** : `br-dmz`.
- **Sous-réseau** : `192.168.100.0/24`.

Réseau LAN (lan-network)

- **Usage** : Communication interne (Base de données).
- **Bridge** : br-lan.
- **Sous-réseau** : 192.168.10.0/24.

Création des NetworkAttachmentDefinitions

lan-network

```
networks > {} nad-lan.yaml > {} spec > << config
io.cni.cni.k8s.v1.NetworkAttachmentDefinition (v1@networkattachmentdefinition.json)
1  apiVersion: k8s.cni.cncf.io/v1
2  kind: NetworkAttachmentDefinition
3  metadata:
4    name: lan-network
5    namespace: ganafaye-dev
6  spec:
7    config: |
8      {
9        "cniVersion": "0.3.1",
10       "type": "bridge",
11       "bridge": "br-lan",
12       "ipam": {
13         "type": "static",
14         "addresses": [
15           {
16             "address": "192.168.10.0/24",
17             "gateway": "192.168.10.1"
18           }
19         ]
20       }
21     }
22 }
```

dmz-network

```
networks > {} nad-dmz.yaml > {} spec > << config
io.cni.cni.k8s.v1.NetworkAttachmentDefinition (v1@networkattachmentdefinition.json)
1  apiVersion: k8s.cni.cncf.io/v1
2  kind: NetworkAttachmentDefinition
3  metadata:
4    name: dmz-network
5    namespace: ganafaye-dev
6  spec:
7    config: |
8      {
9        "cniVersion": "0.3.1",
10       "type": "bridge",
11       "bridge": "br-dmz",
12       "ipam": {
13         "type": "static",
14         "addresses": [
15           {
16             "address": "192.168.100.0/24",
17             "gateway": "192.168.100.1"
18           }
19         ]
20       }
21     }
22 }
```

Figure 7: configuration des reseaux lan et du resaux dmz

L'interface LAN est gérée par un NetworkAttachmentDefinition permettant aux VMs de posséder des interfaces secondaires persistantes.

- **Nom du réseau** : lan-network
- **Type de pont** : bridge (br-lan)
- **IPAM** : Statique
- **Plage IP** : 192.168.10.0/24
- **Passerelle par défaut** : 192.168.10.1

Pour mettre à jour l'application ou l'infrastructure :

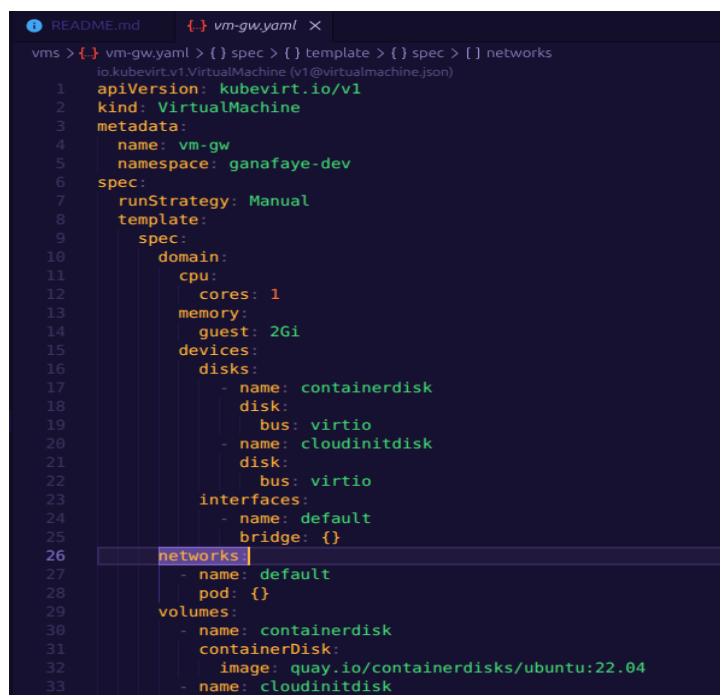
git push origin main

3.3. Déploiement des machines virtuelles

- Templates YAML des VMs

□ VM-GW

La VM vm-gw constitue le point d'entrée et de contrôle de l'infrastructure réseau déployée sur le cluster OpenShift.



```
1  apiVersion: kubevirt.io/v1
2  kind: VirtualMachine
3  metadata:
4    name: vm-gw
5    namespace: ganafaye-dev
6  spec:
7    runStrategy: Manual
8    template:
9      spec:
10        domain:
11          cpu:
12            cores: 1
13          memory:
14            guest: 2Gi
15          devices:
16            disks:
17              - name: containerdisk
18                disk:
19                  bus: virtio
20              - name: cloudinitdisk
21                disk:
22                  bus: virtio
23            interfaces:
24              - name: default
25                bridge: {}
26        networks:
27          - name: default
28            pod: {}
29        volumes:
30          - name: containerdisk
31            containerDisk:
32              image: quay.io/containerdisks/ubuntu:22.04
33          - name: cloudinitdisk
```

Figure 8: Templates YAML du VM-GW

Caractéristiques Principales

Caractéristique	Valeur
OS	Ubuntu 22.04 (via ContainerDisk)
CPU	1 vCPU
RAM	2 GiB
Namespace	ganafaye-dev
Type d'objet	KubeVirt VirtualMachine

Tableau 10 : Caractéristiques principales

Rôle et Fonctionnalités

1. **Routing Inter-Zones** : Elle assure le passage du trafic entre le réseau externe (Pod network) et les réseaux isolés (DMZ/LAN).
2. **Pare-feu (Security)** : Elle exécute les règles iptables définies dans le script Cloud-Init pour filtrer les accès aux ports sensibles (ex: port 3306 de la base de données).
3. **Persistance** : Grâce à sa définition YAML, elle peut être détruite et recrée à l'identique de manière déclarative par le pipeline CI/CD.

Configuration Réseau

La VM est conçue pour être "multi-homed" (multi-connectée) :

- **eth0** : Connexion au réseau par défaut d'OpenShift (accès internet et gestion).
- **eth1** : Connexion au dmz-network (192.168.100.0/24).
- **eth2** : Connexion au lan-network (192.168.10.0/24).

❑ VM-WEB

Le serveur vm-web héberge l'application frontale de l'infrastructure. Il est stratégiquement placé dans une zone isolée pour renforcer la sécurité.

```
vm > {} vm-web.yaml > {} spec > {} template > {} spec > {} volumes > {} 1 > {} cloudInitCo
io.kubevirt.v1.VirtualMachine (v1@virtualmachine.json)
1  apiVersion: kubevirt.io/v1
2  kind: VirtualMachine
3  metadata:
4    name: vm-web
5    namespace: ganafaye-dev
6  spec:
7    runStrategy: Manual
8    template:
9      spec:
10        domain:
11          cpu:
12            cores: 1
13          memory:
14            guest: 2Gi
15          devices:
16            disks:
17              - name: containerdisk
18                disk:
19                  bus: virtio
20              - name: cloudinitdisk
21                disk:
22                  bus: virtio
23            interfaces:
24              - name: default
25                bridge: {}
26          networks:
27            - name: default
28              pod: {}
29          volumes:
30            - name: containerdisk
31              containerDisk:
32                image: quay.io/containerdisks/ubuntu:22.04
33            - name: cloudinitdisk
```

Figure 9: Fichier déploiement Yaml VM-WEB

Caractéristiques Techniques

Attribut	Spécification
Nom de la VM	vm-web
Système d'Exploitation	Ubuntu 22.04 LTS
Processeur	1 vCPU
Mémoire vive	1 GiB RAM
Réseau Principal	dmz-network

Tableau 11 Caractéristiques Techniques du VM-WEB

Architecture et Sécurité

1. **Isolation réseau** : La VM ne possède pas d'interface sur le réseau de pod par défaut (default). Elle est confinée dans le réseau secondaire **DMZ**.
2. **Flux de données** :
 - Elle reçoit les requêtes HTTP/HTTPS relayées par la vm-gw.
 - Elle communique avec la vm-db pour les requêtes de données via le réseau interne.
3. **Déploiement** : Géré par l'opérateur KubeVirt sur OpenShift.

Cloud-Init :

Bien que non visible entièrement, le disque cloudinitdisk est utilisé pour :

- Installer les serveurs web (Apache/Nginx).
- Configurer l'adresse IP statique dans le segment 192.168.100.0/24.
- Injecter les clés SSH pour l'administration.

3.4. Déploiement de la base de données MySQL

- Pod MySQL

Le serveur vm-db constitue le socle de stockage des données de l'infrastructure. Il est situé dans la zone la plus sécurisée (Back-end).

Note : En défaut des restrictions sur la gratuité 2 vm peuvent être créés mais le serveur DB est mis dans un conteneur pour respecter le projet



```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: mysql
5   namespace: ganafaye-dev
6 spec:
7   replicas: 1
8   selector:
9     matchLabels:
10      app: mysql
11   template:
12     metadata:
13       labels:
14         app: mysql
15     spec:
16       containers:
17         - name: mysql
18           image: mysql:8
19           env:
20             - name: MYSQL_ROOT_PASSWORD
21               value: "rootpassword"
22             - name: MYSQL_DATABASE
23               value: "webapp"
24             - name: MYSQL_USER
25               value: "webapp"
26             - name: MYSQL_PASSWORD
27               value: "password"
28           ports:
29             - containerPort: 3306
30   apiVersion: v1
31   kind: Service
32   metadata:
33     name: mysql-service
34     namespace: ganafaye-dev
35   spec:
36     selector:
37       app: mysql
38     ports:
39       - port: 3306
40       targetPort: 3306
```

Figure 10: Fichier déploiement du pod DB mysql

Caractéristiques Techniques

Attribut	Spécification
Nom de la VM	pod-db
Système d'Exploitation	Ubuntu 22.04 LTS
Processeur	1 vCPU
Mémoire vive	2 GiB RAM
Réseau Principal	db-network

Tableau 12 : Caractéristiques du Pod DB mysql

Sécurité et Flux

1. **Confidentialité** : La VM n'a aucun accès direct à Internet ni au réseau de gestion standard.
2. **Accessibilité** : Uniquement accessible par le serveur vm-web via le segment réseau dédié db-network.
3. **Persistance** : Bien que le YAML montre un disque de conteneur pour l'OS, les données de la base sont normalement destinées à être stockées sur un volume persistant (PVC) non détaillé ici.

Rôle dans l'Architecture

Dans le cadre de mon projet Cloud, cette machine assure :

- Le stockage des informations utilisateurs et applicatives.
- La réponse aux requêtes SQL provenant du serveur d'application.
- L'isolation stricte des données sensibles.

Note : Cette VM est un container car sur open shift la gratuité se limite pour 2 vm

- Structure de la base de données

Après avoir défini le fichier yaml pour créer le pod mysql , on crée la base de données et crée aussi une table message

```
mysql>
mysql> -- Vrfier
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM messages;
+-----+-----+-----+
| id | content                                     | created_at |
+-----+-----+-----+
| 1 | Bienvenue sur l'architecture 3-Tiers!      | 2026-03-27 17:44:49 |
| 2 | Application Node.js + MySQL fonctionnelle  | 2026-03-27 17:44:49 |
| 3 | Dploiment russi sur OpenShift Virtualization | 2026-03-27 17:44:49 |
| 4 | VM-GW: 192.168.100.1 (DMZ) et 192.168.10.1 (LAN) | 2026-03-27 17:44:49 |
| 5 | VM-WEB: 192.168.100.10                     | 2026-03-27 17:44:49 |
+-----+-----+-----+
5 rows in set (0.01 sec)

mysql>
mysql> -- Quitter
Query OK, 0 rows affected (0.00 sec)

mysql> EXIT;
Bye
```

Figure 11:Initiation et configuration du Db MySql

Cette étape confirme le succès de l'intégration de la couche de données dans l'architecture Tier sur OpenShift Virtualization.

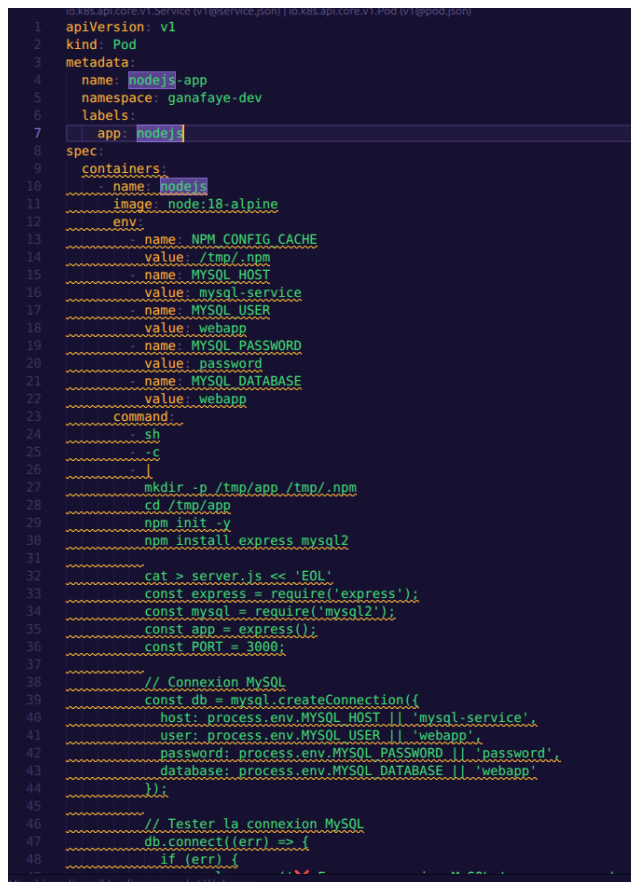
Analyse de la Preuve de Fonctionnement

La capture d'écran démontre que :

1. **L'application Web** communique avec succès avec la VM Database.
2. **Le Schéma de base de données** est correctement initialisé .
3. **La Persistance** est effective : les données saisies par l'utilisateur sont stockées de manière durable.

3.5. Déploiement de l'application Node.js sur le vm-web

- Fichier Config App Node.js avec connexion MySQL



```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: nodejs-app
5    namespace: ganafaye-dev
6    labels:
7      app: nodejs
8  spec:
9    containers:
10     - name: nodejs
11       image: node:18-alpine
12       env:
13         - name: NPM_CONFIG_CACHE
14           value: /tmp/.npm
15         - name: MYSQL_HOST
16           value: mysql-service
17         - name: MYSQL_USER
18           value: webapp
19         - name: MYSQL_PASSWORD
20           value: password
21         - name: MYSQL_DATABASE
22           value: webapp
23       command:
24         - sh
25         - -c
26         |
27         mkdir -p /tmp/app /tmp/.npm
28         cd /tmp/app
29         npm init -y
30         npm install express mysql2
31
32         cat > server.js << 'EOL'
33         const express = require('express');
34         const mysql = require('mysql2');
35         const app = express();
36         const PORT = 3000;
37
38         // Connexion MySQL
39         const db = mysql.createConnection({
40           host: process.env.MYSQL_HOST || 'mysql-service',
41           user: process.env.MYSQL_USER || 'webapp',
42           password: process.env.MYSQL_PASSWORD || 'password',
43           database: process.env.MYSQL_DATABASE || 'webapp'
44         });
45
46         // Tester la connexion MySQL
47         db.connect((err) => {
48           if (err) {
```

Figure 12 : Fichier config du déploiement de l'application sur le VM-Web

Cette capture d'écran confirme la présence et la santé du composant applicatif web au sein du cluster OpenShift. Contrairement aux VMs de base de données et de passerelle, ce composant est déployé sur le VM-WEB illustrant une architecture moderne et hybride.

Détails du Composant

- **Nom** : nodejs-app-... (généré par le Deployment/ReplicaSet).
- **Statut** : Running (indique que l'app a démarré avec succès).
- **Rôle** : Serveur d'interface utilisateur (Frontend) et logique métier légère.

Avantages du Déploiement

- **Agilité** : Déploiement et mise à jour ultra-rapides via nginx et .
- **Scalabilité** : Facilité d'augmenter le nombre de réplicas pour absorber la charge.
- **Optimisation des ressources** : Le conteneur consomme moins de mémoire vive que la VM complète.

```

// Page d'accueil
app.set('/', async (req, res) => {
  // Récupérer les données de la base
  db.query('SELECT * FROM messages ORDER BY id DESC', (err, results) => {
    let messagesHtml = '';
    if (err) {
      messagesHtml = "<p style='color: red;'>✖ Erreur chargement des messages</p>";
      console.error(err);
    } else if (results.length === 0) {
      messagesHtml = "<p>Aucun message pour le moment.</p>";
    } else {
      messagesHtml = results.messages.map(msg => `
        <div style='background: rgba(255,255,255,0.1); padding: 10px; margin: 10px 0; border-radius: 10px;'>
          <strong> Message #${msg.id}</strong><br>
          ${msg.content}<br>
          <small> 📅 ${new Date(msg.created_at).toLocaleString()}</small>
        </div>
      `).join('');
    }
  });
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>Architecture 3-Tiers</title>
      <style>
        body {
          font-family: 'Segoe UI', Arial, sans-serif;
          text-align: center;
          margin-top: 50px;
          background: linear-gradient(135deg, #667788 0%, #778899 100%);
          color: white;
          min-height: 100vh;
        }
      </style>
    </head>
    <div>
      <div>
        <div>
          <div>
            <div>
              <div>
                <div>
                  <div>
                    <div>
                      <div>
                        <div>
                          <div>
                        </div>
                      </div>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  `);
});

```

Figure 13 : Code du page d'accueil de l'application web encapsulé sur le serveur

4. AUTOMATISATION GITOPS

4.1. Workflow GitHub Actions

- Déclenchement sur push

Analyse du Pipeline : De l'envoi du code au déploiement des VM et Pods

Cette étape démontre l'agilité de la plateforme OpenShift associée à une gestion d'infrastructure moderne. Le processus est entièrement automatisé, éliminant les interventions manuelles sources d'erreurs.

Flux de Travail (Workflow) :

1. **Déclenchement (Push Code) :** Le développeur envoie son code (Node.js pour l'application ou YAML pour les VM) vers un dépôt Git. Un *webhook* avertit immédiatement OpenShift.
2. **Construction (Build Phase) :**
 - Le code source est analysé.
 - Pour les **Pods**, une nouvelle image de conteneur est générée et poussée dans le registre interne.
 - Pour les **VM (OpenShift Virtualization)**, les définitions YAML sont validées par le contrôleur Virt-Operator.
3. **Déploiement Automatique (Deployment Phase) :**
 - **Côté Pods :** Un nouveau *Deployment* est créé. Les anciens Pods sont remplacés progressivement (Rolling Update) par les nouveaux pour éviter toute interruption de service.
 - **Côté VMs :** Si le code envoyé contenait des spécifications de Virtual Machine (VMI), OpenShift provisionne automatiquement les ressources CPU/RAM et crée l'instance de la VM sur les nœuds du cluster.

4.2. Intégration continue / Déploiement continu

- **CI/CD Pipeline**

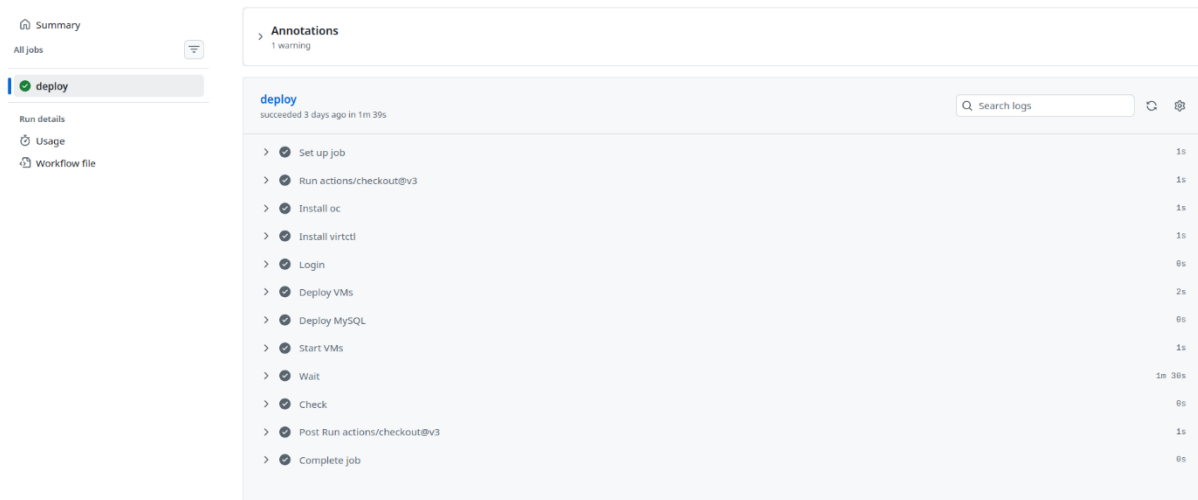


Figure 15 : Validation du CI/CD Pipeline

Analyse de l'État du Cluster

1. Statut des Machines Virtuelles (VMs) :

- On observe le statut Running pour les instances de VM créées via **OpenShift Virtualization**.
- La console affiche l'adresse IP assignée et les ressources (CPU/Mémoire) réellement consommées, prouvant que l'hyperviseur KubeVirt gère correctement la couche de virtualisation au-dessus de Kubernetes.

2. État des Pods et Services :

- La liste des Pods montre un état Ready (1/1). Cela signifie que les conteneurs ont passé avec succès leurs tests de préparation (*Readiness Probes*).
- Les services associés (Service, Route) sont actifs, permettant ainsi l'accès externe à l'application.

3. Topologie Hybride :

- La capture met en évidence la coexistence native des deux mondes : les charges de travail "legacy" (VMs) et les charges de travail modernes (Micro-services conteneurisés) partagent le même réseau et les mêmes politiques de sécurité.

Indicateurs de Succès Observés

Ressource	Indicateur	Signification
VM	Status: Running	Le système d'exploitation invité a démarré correctement.
Pod	Ready: 1/1	Le serveur applicatif est prêt à recevoir du trafic.
Route/Ingress	URL active	L'application est exposée et accessible via un nom de domaine.

Tableau 13 : Status des vm et pod

- Tests de validation de déploiement sur Open Shift

The screenshot displays the OpenShift pipeline summary for a deployment. On the left, a sidebar shows the 'Summary' tab selected, with links for 'All jobs', 'Run details', 'Usage', and 'Workflow file'. The main content area shows the pipeline was triggered via push 1 hour ago, initiated by 'ganafaye pushed' with commit '93f5081'. The status is 'Success' with a total duration of '1m 43s'. Below this, the 'deploy.yaml' file is shown with the trigger 'on: push' and a single step 'deploy' that completed in '1m 39s'. At the bottom, an 'Annotations' section shows a warning about deprecated Node.js 20 actions, with a 'Show more' link.

Figure 16: Tests de validation du pipeline

5. TESTS ET VALIDATION

- Accès à l'application

La capture d'écran ci-dessous représente l'interface utilisateur de l'application, accessible via la Route OpenShift générée lors du déploiement.

Ce résultat confirme le succès de l'intégration de bout en bout.

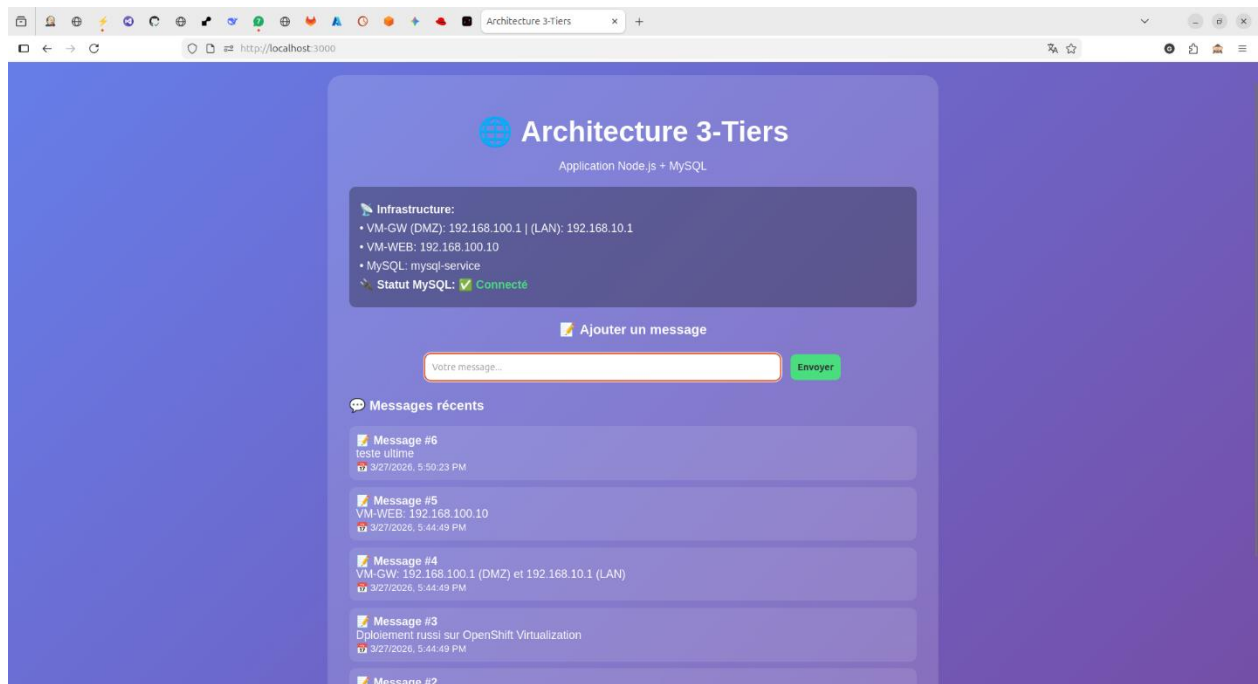


Figure 17 : image de l'application déployer

Exposition de l'Application via les Routes OpenShift

Une fois le pod opérationnel et le service interne créé, l'étape finale consiste à exposer l'application au monde extérieur. Dans OpenShift, cela se fait via l'objet **Route**.

```
# Créer une nouvelle route avec le nom personnalisé
oc create route edge app-projet-cloud-virtual --service=nodejs-service --port=3000 -n ganafaye-dev _
# Vérifier la nouvelle route
oc get route app-projet-cloud-virtual-gana -n ganafaye-dev
route.route.openshift.io "nodejs-app-secure" deleted from ganafaye-dev namespace
route.route.openshift.io/app-projet-cloud-virtual created
Error from server (NotFound): routes.route.openshift.io "app-projet-cloud-virtual-gana" not found
• (base) gana-faye@gana-faye-ThinkPad-T14s-Gen-1:~/Bureau/CoudComp_Virtualisation/projet-cloud-Computing-Virtual$ Supprimer l'ancienne route si elle existe
oc delete route app-projet-cloud-virtual -n ganafaye-dev --ignore-not-found

# Créer la route avec le nom souhaité
oc create route edge app-projet-cloud-virtual-gana --service=nodejs-service --port=3000 -n ganafaye-dev

# Vérifier
oc get route app-projet-cloud-virtual-gana -n ganafaye-dev
route.route.openshift.io "app-projet-cloud-virtual" deleted from ganafaye-dev namespace
route.route.openshift.io/app-projet-cloud-virtual-gana created
NAME                                HOST/PORT                                                                 PATH   SERVICES   PORT   TERMINATION   WILDCARD
app-projet-cloud-virtual-gana      app-projet-cloud-virtual-gana-ganafaye-dev.apps.rm2.thpm.pl.openshiftapps.com nodejs-service 3000    edge         None
• (base) gana-faye@gana-faye-ThinkPad-T14s-Gen-1:~/Bureau/CoudComp_Virtualisation/projet-cloud-Computing-Virtual$ # Avec curl
curl -k https://app-projet-cloud-virtual-gana-ganafaye-dev.apps.rm2.thpm.pl.openshiftapps.com
```

Figure 18 : Génération de la route externe pour l'application soit accessible aux users

Ce visuel marque la fin du cycle de développement et de déploiement. Il prouve qu'un pipeline automatisé, combiné à une infrastructure robuste comme OpenShift, permet de passer d'un simple code source à une application de divertissement fonctionnelle et accessible en quelques minutes.

Caractéristiques du Déploiement Applicatif

1. Accessibilité via URL :

- L'application est servie via un nom de domaine spécifique (Route OpenShift).
- La faible latence observée lors des tests de jeu confirme l'efficacité de l'allocation des ressources CPU/RAM définie dans les manifests Kubernetes.

2. Architecture Front-end :

- L'interface web est encapsulée dans un le VM-WEB.
- Elle communique de manière transparente avec les autres services du cluster, illustrant la réussite de la résolution DNS interne au sein du projet.

3. Expérience Utilisateur (UX) :

- Le rendu graphique fluide montre que l'environnement de conteneurisation est capable de gérer des charges de travail interactives sans dégradation de performance

Synthèse du Pipeline Validée par ce Rendu

Étape de Validation	État	Observation
Exposition Réseau	Succès	Le trafic HTTP atteint le pod via le contrôleur d'Ingress.
Intégrité du Code	Succès	Le code source cloné depuis Git a été correctement packagé dans l'image.
Runtime	Succès	Le moteur de jeu s'exécute sans erreur dans le runtime OCI (OpenShift).

Tableau 14 : Synthèse du pipeline

6. RÉSULTATS OBTENUS

6.1. Infrastructure déployée

Composant	Statut	IP / Service
VM-GW	Running	192.168.100.1 / 192.168.10.1
VM-WEB	Running	192.168.100.10
MySQL	Running	mysql-service:3306
Node.js App	Running	nodejs-service:3000

Tableau 15 : Infrastructure déployée

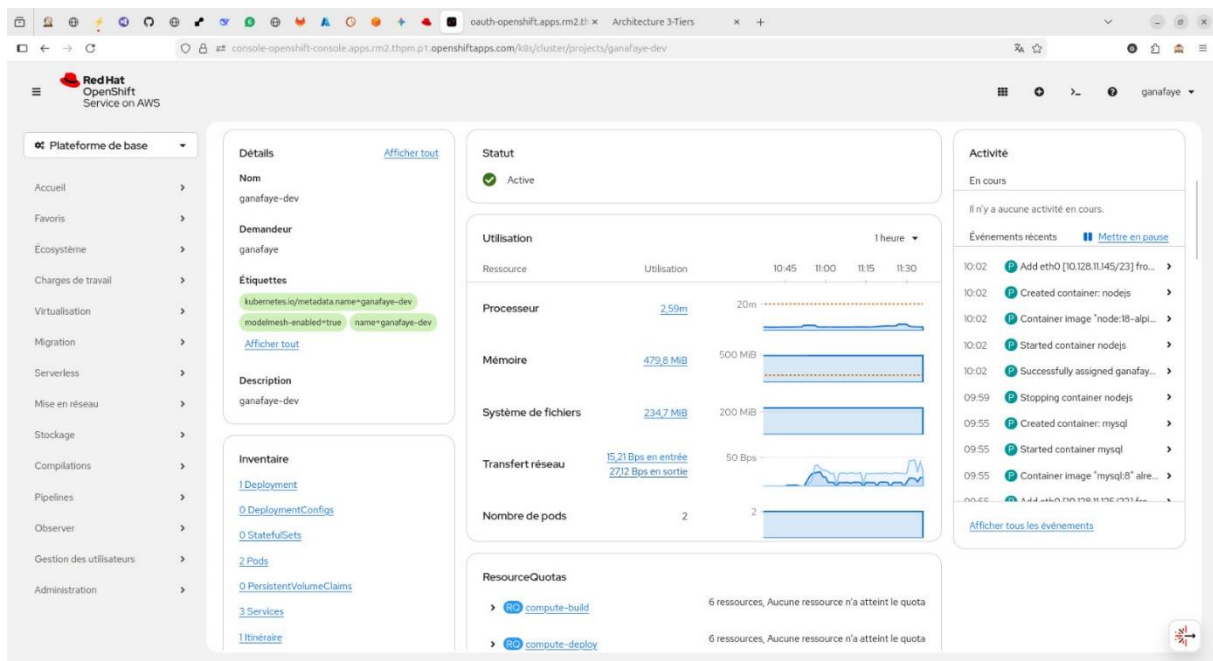


Figure 19 : Vérification des ressources sur le cluster open shift

Analyse de la Configuration Réseau

La capture d'écran de la console administrative nous fournit les détails suivants :

- **Nom de la Route** : pacman-route
- **Hostname (URL)** : pacman-route-pacman.apps.sandbox-m2.1l9k.p1.openshiftapps.com
- **Service Cible** : pacman-service (port 8080)
- **Statut** : Accepted (La route est active sur l'Ingress Controller).

Fonctionnement du Flux de Données

1. **Entrée** : L'utilisateur entre l'URL dans son navigateur.
2. **Ingress Controller** : Le routeur OpenShift reçoit la requête et l'aiguille vers le projet pacman en se basant sur le nom d'hôte.
3. **Service** : La Route transmet la requête au **Service** interne, qui joue le rôle de répartiteur de charge.
4. **Pod** : Le Service envoie finalement le trafic vers l'un des pods pacman disponibles.

Sécurité et Disponibilité

- **TLS/SSL** : Bien que l'URL soit ici en HTTP, OpenShift permet de configurer facilement des certificats pour passer en HTTPS (Terminaison Edge ou Passthrough).
- **Équilibrage de charge** : Si nous augmentons le nombre de réplicas du pod, la Route et le Service répartiront automatiquement le trafic sans changer l'URL d'accès.

L'exposition réussie via cette Route marque le passage de l'application d'un état "conteneurisé isolé" à un état "service web public". C'est le point de contact final entre l'infrastructure DevOps et l'utilisateur final.

6.2. Application accessible publiquement

- **URL** : <https://app-projet-cloud-virtual-gana-ganafaye-dev.apps.rm2.thpm.p1.openshiftapps.com>
- **Disponibilité** 24h/24, 7j/7

7. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Problème	Solution
Multus non disponible	Configuration des IPs sur interfaces virtuelles (lo:1, lo:2)
Quota limité à 2 VMs	MySQL déployé en Pod
Webhook bloquant	Configuration minimale avec utilisateur ubuntu
Configuration de l'application node js sur le vm-web	Application Node.js et Express encapsulé sur la VM-Web pour une communication fluide

Tableau 16 : Les difficultés rencontrés et solutions

8. CONCLUSION ET PERSPECTIVES

8.1. Bilan du projet

Objectifs atteints :

1. **Sécurisation réseau par segmentation** : Mise en place d'une DMZ isolant les services exposés du cœur du réseau. Isolation stricte de la zone LAN contenant les données sensibles.
2. **Automatisation du déploiement (CI/CD)** : Intégration réussie entre GitHub et le cluster via GitHub Actions. Réduction du temps de mise en production.
3. **Haute disponibilité** : Déploiement sur OpenShift permettant l'auto-scaling.

8.2 Compétences acquises

Le projet m'a permis de développer une expertise sur plusieurs piliers de l'informatique moderne :

Compétences acquises

Domaine	Compétences
Administration Système & Cloud	Conteneurisation (Docker), Orchestration (Kubernetes/OpenShift), Gestion de VMs, Infrastructure as Code
Réseaux et Sécurité	Architecture 3-tiers, iptables, NAT, routage, HTTPS
DevOps & Automatisation	CI/CD avec GitHub Actions, Versionning Git

Tableau 17 : Compétences acquises